

Technical Reflection Report

Global Logistics Management System (GLMS)

TechMove Logistics — ASP.NET Core MVC + Web API

1. DevOps & Testing: Automated Testing in a CI/CD Pipeline

Why Automated Testing is Critical

In a CI/CD (Continuous Integration / Continuous Deployment) pipeline, code is integrated and deployed multiple times per day. Without automated tests, every deployment is a leap of faith. A developer could introduce a breaking change — a null reference error in the contract validation logic, a bad currency conversion formula, or a broken API endpoint — and that change could silently reach production before anyone notices.

Automated testing prevents this by acting as a **quality gate**: before any code is promoted to the next stage (dev → staging → production), the test suite runs automatically. If any test fails, the pipeline stops and the deployment is blocked. This creates a feedback loop measured in seconds, not days.

How Tests Prevent Bugs Reaching Production

In the GLMS project, three layers of automated testing were implemented:

Unit Tests (GLMS.Tests — Pre-existing)

Test Class	What it guards
CurrencyCalculationTests	Verifies the USD→ZAR conversion formula is mathematically correct at known values (100 USD × 18.50 = R1 850.00). A regression here would silently over- or under-charge clients.
FileValidationTests	Verifies that only `.pdf` files are accepted for signed agreements. A bug here could allow malicious file uploads.
ContractWorkflowTests	Verifies that service requests cannot be created on `Expired` or `OnHold` contracts. A regression here would violate a core business rule.

These tests are **pure logic tests** — they run in milliseconds, have no external dependencies, and catch regressions in the calculation and validation code immediately.

Integration Tests (GLMS.Tests.Integration — Added in Part 3)

The integration tests go further: they spin up a real in-memory copy of the GLMS Web API using `WebApplicationFactory<ApiMarker>`, seed it with test data, and make actual HTTP calls to assert the API behaves correctly end-to-end.

Key scenarios covered:

`GET /api/contracts` returns 200 OK with a non-null JSON body.

`GET /api/contracts?status=Active` returns only Active contracts — filtering logic is verified at the HTTP layer.

`POST /api/servicerequests` with an Expired contract returns 400 Bad Request — the business rule is enforced all the way from the endpoint down to the database query.

`PATCH /api/contracts/{id}/status` with a non-existing ID returns 404 Not Found.

These tests catch an entire class of bugs that unit tests cannot: routing errors, serialisation failures, middleware misconfiguration, and EF Core query mistakes. They are the last automated defence before a human tester or an end user encounters a bug.

The CI/CD Gate in Practice

In a GitHub Actions or Azure DevOps pipeline, the workflow looks like this:

Developer pushes code



CI pipeline triggers



dotnet build ← catches compile errors



dotnet test ← 59 tests run (28 unit + 31 integration)



If ALL pass → deploy to staging

If ANY fail → pipeline blocked, developer notified immediately

This means a developer cannot accidentally deploy a broken contract validation rule or a broken API endpoint. The tests act as **executable documentation** of what the system is supposed to do, and they enforce that contract on every single commit.

2. Containerisation: Docker and the "Works on My Machine" Problem

The Problem Docker Solves

Software development historically suffered from a critical problem: code that worked perfectly on a developer's Windows 11 machine would fail in production on a Linux server. The root causes were subtle but damaging:

- Different versions of the .NET runtime installed
- Different SQL Server versions or configurations
- Different environment variables or file paths
- Missing dependencies that were installed manually and never documented

This produced the infamous phrase: "**It works on my machine.**"

Docker solves this by packaging the application together with **everything it needs to run** — the runtime, the web server, the configuration, the file system structure — into a single portable image. When you build and run a Docker image, you get an identical environment every time, on every machine, on every operating system.

How GLMS Uses Docker

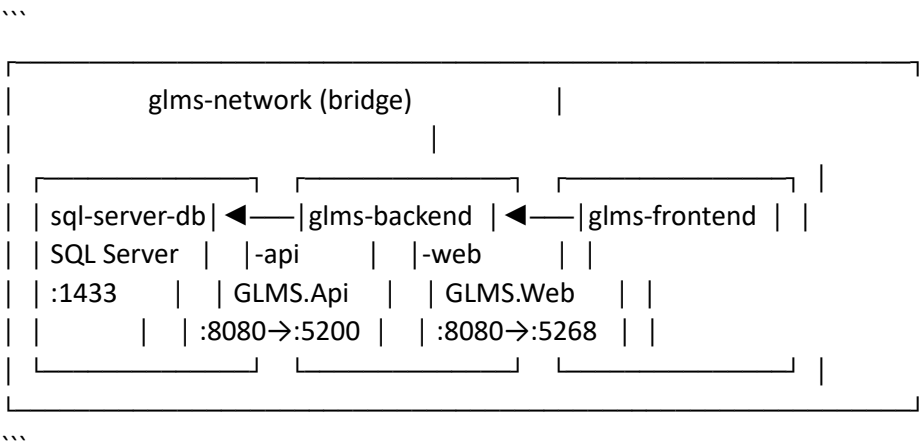
The GLMS system is containerised as a **three-container architecture** using Docker Compose:

Container	Image	Role
sql-server-db	mcr.microsoft.com/mssql/server:2022-latest`	Persistent SQL Server database. Data is stored in a named Docker volume (sqlserver_data) so it survives container restarts.
glms-backend-api	Built from GLMS.Api/Dockerfile	The REST API. Connects to sql-server-db` via Docker's internal DNS. Auto-applies EF Core migrations on startup. Exposes Swagger UI at `:5200/swagger`.

glms-frontend-web

Built from GLMS.Web/Dockerfile

The MVC frontend. Connects to glms-backend-api via Docker's internal DNS (http://glms-backend-api:8080). Exposes the browser UI at `:5268`.



Multi-Stage Builds

Both Dockerfiles use **multi-stage builds** — a Docker best practice that keeps the final image small and secure:

- 1. **Build stage** uses the full .NET SDK image to compile and publish the application.
- 2. **Final stage** uses only the lightweight ASP.NET runtime image — no compiler, no source code, no build tools.

The result is a production image that is typically 60–80% smaller than a single-stage build, with a significantly reduced attack surface.

Environment Consistency Across Dev, Test, and Prod

Environment	How GLMS runs	Database
Dev (local)	`dotnet run` on Windows	SQL Server Express on host via Windows Auth
Test (CI pipeline)	`dotnet test`	EF Core InMemory — no external DB needed
Docker (Prod-like)	`docker compose up`	SQL Server container, SA authentication

The key insight is that the application code is **identical** in all three environments. Only the configuration changes — the connection string and the API base URL are injected via environment variables in Docker Compose. This means a bug fixed locally will be fixed in production: the same binary, the same runtime, the same behaviour.

Dependency Ordering and Health Checks

A subtle but important Docker Compose feature used in GLMS is the **health check** on the SQL Server container:

```
yaml
healthcheck:
  test: sqlcmd -S localhost -U sa -P "..." -Q "SELECT 1" -No
  interval: 15s
  retries: 10
  start_period: 45s
```

The `glms-backend-api` service has `depends_on: sql-server-db: condition: service_healthy`. This means the API container will not start until SQL Server is confirmed to be accepting connections. Without this, the API would crash on startup because the database isn't ready yet — a common "works locally, breaks in Docker" failure mode.

Summary

Docker transforms GLMS from a solution that only works on one configured Windows machine into a portable, reproducible system that any team member, CI server, or cloud environment can run with a single command:

```
```bash
docker compose up --build
```

This is the foundation of modern cloud-native deployment — the application is environment-agnostic, infrastructure is code, and "it works on my machine" becomes "it works everywhere."